

计算机系统导论实验报告

**实验名称: Switch 语句与链接
(IA-32)**

班级: 计算 2403

姓名: 钟智勇

日期: 2026 年 1 月 7 日

目录

1 Switch 语句与链接 (IA-32)	3
1.0.1 1. 实验任务与目标	3
1.0.2 2. 完成该实验需要掌握的知识	3
1.0.3 3. 实验过程记录	3
1.0.3.1 3.1 提取 COOKIE 字符串	3
1.0.3.2 3.2 分析 Switch 索引逻辑	3
1.0.3.3 3.3 建立目标映射关系	3
1.0.3.4 3.4 修改跳转表	4
1.0.4 4. 遇到的问题与解决过程	4
1.0.5 5. 心得体会	4

1 Switch 语句与链接 (IA-32)

1.0.1 1. 实验任务与目标

本实验的任务是修改一个名为 `phase1.o` 的二进制可重定位目标文件。该模块中的 `do_phase()` 函数会处理一个内置的 `COOKIE` 字符串，并通过 `switch` 语句对字符进行变换。-目标：在不修改 `.text`（代码节）和重定位节的前提下，通过修改 `.rodata`（只读数据节）中的跳转表（Jump Table）内容，使得程序与 `main.o` 链接运行后，能够输出指定的目标字符串“123456789”。

1.0.2 2. 完成该实验需要掌握的知识

- Switch 的跳转表机制：当 `switch` 语句的 `case` 值较为连续时，编译器会生成一个地址数组（跳转表）。程序通过 `Base + Index * 4` 的方式定位目标代码地址。
 - ELF 文件结构：了解可重定位目标文件（.o）的节（Section）布局，特别是 `.rodata` 节通常存放跳转表。
 - 重定位（Relocation）：链接器在合并模块时，会根据重定位表修改占位符地址。在 .o 文件中，跳转表里的地址往往是相对于节首的偏移量。
 - COOKIE 字符映射：需理解如何将输入的字符（如 'G'）转换为跳转表的索引（如 `Index = 'G' - 'A'`）。
-

1.0.3 3. 实验过程记录

1.0.3.1 3.1 提取 COOKIE 字符串

通过分析 `do_phase` 的反汇编代码，我首先还原了程序处理的原始 `COOKIE`。

```
2e: c7 45 e6 47 43 5a 56    movl    $0x565a4347, -0x1a(%ebp) ; 'G','C','Z','V'
35: c7 45 ea 49 50 48 58    movl    $0x58485049, -0x16(%ebp) ; 'I','P','H','X'
3c: 66 c7 45 ee 42 00      movw    $0x42, -0x12(%ebp) ; 'B'
```

由此得出 `COOKIE` 字符串为：GCZVIPHXB（共 9 位，与目标字符串长度一致）。

1.0.3.2 3.2 分析 Switch 索引逻辑

代码 60 处执行了 `sub $0x41, %eax`，其中 `0x41` 是字符 'A'。这意味着跳转表的索引是基于字符与 'A' 的距离计算的：

- 'G' 对应索引
- 'C' 对应索引
- 'B' 对应索引

1.0.3.3 3.3 建立目标映射关系

我的目标是让 `COOKIE` 的第 `i` 个字符跳转到能输出数字的 `case` 分支。通过查看 `do_phase` 的分支代码，我找到了输出各数字的指令地址偏移：

- 输出 '1'：地址 `0xb1` (`movb $0x31, -0x9(%ebp)`)
- 输出 '2'：地址 `0xe1` (`movb $0x32, -0x9(%ebp)`)
- 输出 '3'：地址 `0xff` (`movb $0x33, -0x9(%ebp)`)
- ... (依此类推至 '9'：地址 `0x7e`)

接下来，我建立了 `COOKIE` 字符与目标地址的对应表：

1. 'G' (Index 6) -> 需跳转至 0xb1 (输出 1)
2. 'C' (Index 2) -> 需跳转至 0xe1 (输出 2)
3. 'Z' (Index 25) -> 需跳转至 0xff (输出 3)
4. 'V' (Index 21) -> 需跳转至 0x9f (输出 4)
5. 'I' (Index 8) -> 需跳转至 0x105 (输出 5)
6. 'P' (Index 15) -> 需跳转至 0x117 (输出 6)
7. 'H' (Index 7) -> 需跳转至 0xd5 (输出 7)
8. 'X' (Index 23) -> 需跳转至 0xe7 (输出 8)
9. 'B' (Index 1) -> 需跳转至 0x7e (输出 9)

1.0.3.4 3.4 修改跳转表

使用 `readelf -S phase1.o` 确定 `.rodata` 节的文件偏移。跳转表就存储在该节中。我使用十六进制编辑器 (如 `hexedit`)，根据上面推导出的索引位置，修改对应的 4 字节地址值。由于是小端序，例如将索引 6 的位置修改为 `b1 00 00 00`。

修改完成后，执行链接命令：`gcc -no-pie -o linkbomb main.o phase1.o` 运行 `./linkbomb`，程序成功输出 123456789。

1.0.4 4. 遇到的问题与解决过程

- 问题：跳转表位置找不准。反汇编中 `mov 0x4(,%eax,4), %eax` 里的 `0x4` 让我很困惑。
 - 解决：通过 `readelf -r phase1.o` 查看重定位表，我发现有一项 `R_386_32` 指向 `.rodata`。这证实了跳转表确实从 `.rodata` 偏移为 4 的位置开始。每一个项占用 4 字节，通过 `Index 4 + 4` 就能准确定位每一个字符对应的跳转地址槽位。
-

1.0.5 5. 心得体会

- 链接的本质：以前觉得链接只是把文件拼在一起，现在明白链接过程实际上是在打“补丁”。我们在 `.o` 文件里改掉跳转表，链接器就会把这些改动带入到最终的可执行文件中。
- 数据引导逻辑：这个实验展示了代码逻辑如何被数据（跳转表）所驱动。通过修改数据而非指令，我们同样能彻底改变程序的执行流。这让我对“数据与代码边界模糊”的底层逻辑有了更真实的体会。
- 逆向工具的配合：`objdump` 看逻辑，`readelf` 找位置，十六进制编辑器做手术。这三者的配合是处理二进制文件的基本功。